

INVESTIGACIÓN

Diseño a Código Asistido por IA

Figma Make · MCP Server · Claude Code · Angular 18

Ginés López Montalbán

Product Designer — UX/UI & Frontend Development

Julio 2026

Figma Make	MCP Server	Claude Code	Angular 18	Design Systems	Tokens Semánticos	Living Spec	Flujo IA	Arquitectura de Componentes
------------	------------	-------------	------------	----------------	-------------------	-------------	----------	-----------------------------

Resumen

Investigación sobre flujos de trabajo asistidos por IA que conectan tres capas tecnológicas — Figma Make, el servidor MCP de Figma y Claude Code — para generar componentes y vistas Angular listos para producción a partir de especificaciones de diseño. La investigación utilizó una pantalla de resultados de análisis de sangre como escenario, cubriendo el pipeline completo desde la definición del design system hasta la implementación en Angular 18. La investigación produjo una metodología reutilizable: documentos de contexto, bibliotecas de prompts, plantillas de especificación de componentes y un conjunto de reglas derivadas empíricamente para proyectos futuros.

Objetivos de la investigación

Evaluar el ecosistema Figma AI	Valorar las capacidades prácticas y limitaciones de Figma Make, el servidor MCP y su integración con Claude Code para flujos de desarrollo frontend Angular/Ionic.
Validar el pipeline diseño a código	Documentar la cadena completa desde la creación del design system en Figma Make hasta la generación de componentes Angular 18 via Claude Code y MCP Server.
Definir una metodología reutilizable	Producir documentos de contexto estructurados, plantillas de prompts y estándares de especificación de componentes aplicables a proyectos futuros de forma sistemática.
Identificar puntos de mejora	Documentar puntos de fricción, antipatrones y decisiones de arquitectura que emergen durante la investigación, convirtiéndolos en reglas para futuras iteraciones.

Establecer responsabilidades entre capas

Definir límites claros entre Claude.ai (orquestración), Figma Make (diseño y especificación) y Claude Code (implementación) para eliminar ambigüedad en el proceso.

Metodología

Fase 0	Style guide	Tokens de color, tipografía y espaciado definidos como frames de referencia visual en Figma Make.
Fase 1	Componentes	Tres componentes con cabeceras living spec: InputText, Button, CardResult con variantes de estado clínico.
Fase 1.5	Tokens espaciado	Escala de espaciado aplicada a todos los componentes antes de construir las vistas.
Fase 2	Vistas	Dos variantes de pantalla responsiva generadas usando instancias de componentes, no divs redibujados.
Fase 2.5	Revisión y ajuste	Edición directa de código y refinamiento por prompts en el sandbox de Make antes de exportar.
Fase 3	Exportación Angular	Exportación en dos fases via Claude Code + MCP: análisis con warnings, luego generación tras confirmación.
Fase 3C	Verificación final	Compilación, revisión de imports, validación de tokens SCSS y verificación visual del render.

Hallazgos principales

- 01

Figma Make genera código de renderizado, no código de producción

Los componentes usan estilos inline, props literales (colores hex, valores de opacidad) y elementos HTML no semánticos como divs en lugar de buttons o inputs. Es el comportamiento esperado de un sandbox de prototipado — pero requiere una estrategia de traducción estructurada a Angular.
- 02

La cabecera living spec es el puente crítico

Definir una cabecera de especificación técnica estructurada (inputs, variantes, tokens, layout, semántica HTML) directamente en el canvas de Figma Make antes de la representación visual demostró ser el mecanismo más eficaz para guiar a Claude Code hacia un output Angular correcto. La cabecera tiene prioridad sobre el visual y sobre el código React.
- 03

La estrategia de exportación en dos fases reduce errores significativamente

Separar el análisis (con reporte explícito de warnings) de la generación — y requerir confirmación entre fases — detectó 12 warnings en componentes y vistas antes de escribir una sola línea de código Angular. Un warning detectado antes de generar cuesta un mensaje; el mismo problema después de generar puede costar una hora de refactorización.
- 04

Los tokens semánticos no son negociables

Los componentes que reciben color como dato de instancia (accent, bg, border como strings hex) crean complejidad de traducción que no existe si el componente usa variantes semánticas desde el origen. La regla 'los colores nunca son valores @Input()' debe aplicarse como restricción de diseño, no de code review.

05

Figma Make no instancia componentes en las vistas — los redibuja

Las vistas generadas en Make usaron divs genéricos que visualmente se parecen a los componentes definidos, no instancias reales. Esto rompe la cadena de composición y genera ambigüedad de layout en la exportación. Se requieren instrucciones explícitas en el prompt para forzar a Claude Code a importar los componentes existentes en lugar de regenerarlos.

06

La potencia real del ecosistema Figma está en una cadena diferente

La investigación reveló que Figma Make es una herramienta de prototipado rápido, no un puente diseño-a-código. La cadena con mayor potencial para Angular es: Figma Design (Variables y componentes estructurados) → MCP → Code Connect → Claude Code. Code Connect mapea instancias de componentes Figma directamente a componentes Angular con sus imports y prop interfaces correctos, eliminando el problema de traducción por completo.

Conclusiones

- Figma Make acelera significativamente la iteración visual, pero requiere disciplina estructurada (cabeceras living spec, arquitectura de tokens semánticos, verificación de checklist) para producir un output que Claude Code pueda traducir a Angular con mínima intervención manual.
- La estrategia de exportación en dos fases (análisis → confirmación → generación) es una barrera de calidad práctica que debería ser estándar en cualquier flujo de generación de código asistido por IA, independientemente de la herramienta de diseño utilizada.
- El principio de ambigüedad se aplica universalmente: cualquier cosa que no esté explícitamente especificada será decidida por la herramienta de IA con sus propios criterios. La calidad del output es directamente proporcional a la especificidad del contexto de entrada.
- La investigación produjo una metodología reutilizable: cinco documentos estructurados que cubren responsabilidades del sistema, contexto de orquestación de Claude.ai, checklist de arranque de proyecto, plantilla de contexto para Figma Make y plantilla de contexto para Claude Code.
- El siguiente paso lógico — Figma Design con Code Connect — eliminaría el problema de traducción de raíz, exponiendo especificaciones de componentes estructuradas directamente a Claude Code via MCP, en lugar de código React de renderizado que requiere interpretación.
- Conclusión de viabilidad: la investigación confirma que es posible diseñar en Figma Make y exportar estilos, tokens y componentes directamente a Angular con mínima intervención, respetando el procedimiento definido. El flujo funciona de extremo a extremo — desde la definición del design system hasta la aplicación Angular compilada y en ejecución — y produce código mantenible cuando se aplican las reglas de especificación correctamente. La intervención humana se concentra en la toma de decisiones de diseño y en la revisión de warnings antes de la generación, no en la traducción manual de diseño a código.

Entregables de la investigación

Informe de evaluación

Documento que cubre la investigación completa: objetivos, descripción de capas, prompts, resultados de ejecución, warnings y conclusiones estratégicas.

figma_make_context.md	Documento de contexto reutilizable para proyectos en Figma Make: reglas de arquitectura de tokens, orden de construcción de componentes, plantilla living spec y checklist de antipatrones. Incluye correcciones v1.1 derivadas de la investigación.
DOC 01 — Responsabilidades del sistema	Definición de la arquitectura de tres capas (Claude.ai / Figma Make / Claude Code), sus responsabilidades, qué necesita saber cada capa y la jerarquía de autoridad entre ellas.
DOC 02 — Contexto inicial de Claude.ai	Instrucciones de orquestación y checklist de arranque de proyecto en seis bloques, listo para cargar como contexto en cualquier proyecto nuevo.
Proyecto Angular	Aplicación Angular 18 funcional con componentes standalone (InputText, Button, CardResult), vista de resultados responsive, arquitectura de tokens semánticos en SCSS y routing con lazy loading — generada via Claude Code + MCP desde el archivo Figma Make.

Esta investigación confirma la viabilidad del pipeline diseño-a-código asistido por IA: es posible diseñar en Figma Make y exportar componentes directamente a Angular. La clave está en la disciplina de especificación, no en las herramientas.